

**Author: Isaac Miller**

## **General Disclosure**

GAI Tool used: Claude

GAI Model: Sonnet 5

Dates Tool was Consulted: 9/24, 9/25, 9/27

## **Code Attribution**

Prompt 1:

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
bool running = true;
```

```
int extractIPv4(const char* str, unsigned long* outAddress, int* outPort) {
```

```
    ;
```

```
};
```

```
int main() {
```

```
    char input[100];
```

```
    unsigned long outAddress;
```

```
    unsigned long decimal;
```

```
    int outPort;
```

```
    while (running) {
```

```
        printf("Enter a string (or 'END' to quit): ");
```

```
        if (scanf("%29s", input) != 1) {
```

```
            printf("Invalid input: no valid IPv4 address found\n");
```

```
        } else if (input == 'END') {
```

```

        running = false;

    } else {

        if (extractIPv4(input, 0, -1) == 1) {

            printf("Extracted IPv4 address: %s (decimal value: %d, port: %d)\n", outAddress,
decimal, outPort);

        } else {

            printf("Invalid input: no valid IPv4 address found\n");

        }

    }

}

printf("Program terminated.");

}

```

Good evening! I'm working on a program that takes a string, finds an IP address (and optionally a port number), and returns the IP address as well as its decimal form (and optional port number). Could you help me set this up? Its important to note that this shouldn't actually query for a IP address, but simulates the extraction of the IP address.

I put in this prompt with the starter code that I had made as a sort of guide point, so that Claude knew how I wanted my code to be organized as well as what I wanted it to help me with. This allowed Claude to create the extractIPv4() function very closely with how I wanted it to be.

Prompt 2:

Sweet! That works as intended! Another thing is that when I enter "Connecting to 192.168.1.1 now" (which should return and pass through) it states this:

```
Enter a string (or 'END' to quit): connecting to 192.168.1.1 now
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): Invalid input: no valid IPv4
address found
Enter a string (or 'END' to quit): Extracted IPv4 address: 192.1
68.1.1 (decimal value: 3232235777, no port found)
Enter a string (or 'END' to quit): Invalid input: no valid IPv4
address found
Enter a string (or 'END' to quit):
```

I noticed this output when I was testing some of my test cases with multiple words in each string. This was happening because my previous `scanf()` call was creating four separate reads for each word in the string above. The solution to this was to use `fgets()`, which reads and passes the whole line into `extractIPv4` instead of passing four different strings into it.

#### Prompt 3:

Awesome! There's a couple more requirements that I'll need to double check. The first is that we shouldn't address leading zeroes in the IP string or Port number, so any leading zeroes present should invalidate the IP address

In my original prompt, I never specified the “leading zero” rule. So I used this prompt to have Claude help me implement the “leading zero” rule into the `extractIPv4()` function. It checks if there's a leading zero by checking if there's more than one digit in the octet, and then checking if the first digit is a 0. If both of these are true, the match is rejected.

#### Prompt 4:

Yes, please add that boundary check at the to catch that, that would be awesome!

This is in reference to a boundary check where the parser would ignore previous ‘s and digits that were before (I.E. “1.2.3.4.5” would pass as “2.3.4.5” when it shouldn't pass). So, Claude helped me implement a check where a match can't start right after a digit or a ‘. In `extractIPv4()`.

#### Prompt 5:

Could you help me come up with a variety of test cases to make sure that I catch all of the checks that we have implemented in this program?

This prompt was used to help come up with all of the test cases in my test cases document. I made this prompt to help me capture the edge cases of the previous conditions and requirements that I had previously listed, and this proved incredibly helpful as I had missed a check where if there weren't any digits after the ":", the program would still process the IP address before the ":". Checking these test cases helped me verify that my program worked as intended and spot bugs and unexpected output that I had missed previously.

#### Parts of Code that were student-written vs. AI-generated

There were two checks in `extractIPv4()` that I implemented myself (one was to check for trailing periods after the IP address and the other was to ensure that matches are rejected if there is no port number [previously accepted "10.0.0.1:" and "10.0.0.1:abc"]), but Claude helped write every other part of the `extractIPv4` method. The `main()` functions "skeleton" was written by me, but Claude also helped fix an error where `scanf()` was reading multiple strings per input (fixed with `fgets`) and helped modify the if statements after `extractIPv4()` was called so that it produced the desired `printf()` statements that I wanted.

#### Modifications to AI-generated output:

As mentioned previously, I implemented a check to see that there was no trailing period after an IP address match so that matches such as "1.2.3.4.". This was done in an if statement's condition, where I specified that the cursor pointer was not pointing at another period. I also implemented my own check when checking for valid ports, since the program didn't reject empty space or strings after a colon following an IP such as "10.0.0.1:" and "10.0.0.1:abc". I fixed this by making sure the variable `digits` was not equal to zero, meaning there was no port number following the colon.

#### **Verification Statement:**

I can confirm that I understand every line of code in this program. Each block of code will be commented on and summarized to help prove this point.

After testing the code with the test cases in my test cases document, I can confirm that the program works as I intended it to.

To my knowledge, there are no bugs, limitations, or unexpected behavior from this program after checking with each of the test cases in the testing document.